Chapter 10. Attention and Transformers

Lectures 11/2, 11/4: attention-based captioning, spatial and channel attention, inductive bias, scaled dot-product, cross- and self-attention, positional encoding, masking, multi-head attention, Transformer encoder, linearized attention, layer normalization, Vision Transformer, GELU, Swin Transformer, precipitation nowcasting

Contents

1 Attention-Based Image Captioning **2**

2 Spatial and Channel Attention **2**

3 Inductive Bias **3**

4 The General Attention Layer **4**

4.1 One query 4

4.2 Why divide by $\sqrt{D}$ 4

4.3 Many queries, keys and values 4

5 Permutation Symmetry **5**

6 Positional Encoding **5**

7 Masked Self-Attention **6**

8 Multi-Head Attention **7**

9 The Transformer Encoder **7**

10 Linearization of the Transformer **8**

11 Layer Normalization **9**

12 Why Transformers **10**

13 Vision Transformer **10**

13.1 GELU 11

13.2 ViT versus CNN 11

14 Swin Transformer (Advanced) **11**

15 Application: Radar-Based Precipitation Nowcasting **12**

1 Attention-Based Image Captioning

Motivation. Each generated word re-weights the image regions it describes.

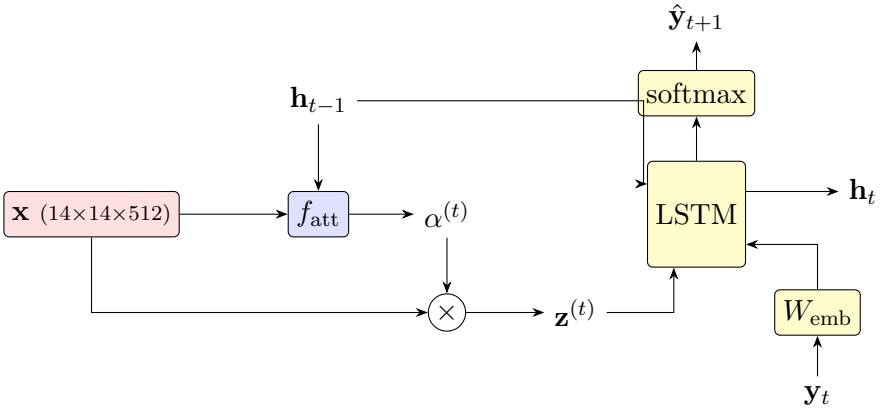
Keep VGG's conv5 map $\mathbf{x} \in \mathbb{R}^{14\times14\times512}$ ($224 \times 224 \times 3$ input): $L = 196$ vectors $\mathbf{x}_{i,j} \in \mathbb{R}^{512}$. Each word t has an **attention map** $\alpha^{(t)} \in \mathbb{R}^{14\times14}$, nonnegative with $\sum_{i,j} \alpha^{(t)}_{i,j} = 1$ ($\alpha^{(1)}$ for "a", $\alpha^{(2)}$ for "bird", ...), summarizing the image as $\mathbf{z}^{(t)} = \sum_{i,j} \alpha^{(t)}_{i,j}\mathbf{x}_{i,j} \in \mathbb{R}^{512}$.

Definition 10.1: attention-based captioning (Show, Attend and Tell)

At step t, with previous hidden state $\mathbf{h}_{t-1}$ and current input word $\mathbf{y}_t$:

$$
\begin{aligned}
e^{(t)} &= f_{\text{att}}(\mathbf{x}, \mathbf{h}_{t-1}) && \text{(alignment scores, one per location)},\\
\alpha^{(t)}_{i,j} &= \frac{\exp(e^{(t)}_{i,j})}{\sum_{i',j'} \exp(e^{(t)}_{i',j'})} && \text{(attention weights)},\\
\mathbf{z}^{(t)} &= \sum_{i,j} \alpha^{(t)}_{i,j}\mathbf{x}_{i,j} && \text{(context vector)},\\
\mathbf{h}_t &= \text{LSTM}(\mathbf{h}_{t-1}, W_{\text{emb}}\mathbf{y}_t, \mathbf{z}^{(t)}) && \text{(modified LSTM)},\\
\hat{\mathbf{y}}_{t+1} &\propto \exp\left(W^o(W_{\text{emb}}\mathbf{y}_t + W^h\mathbf{h}_t + W^z\mathbf{z}^{(t)})\right) && \text{(next-word distribution)}.
\end{aligned}
$$

The context vector enters every LSTM gate; f_{att} is a small MLP; the cell repeats once per word.



$(\mathbf{x}, \mathbf{h}_{t-1}) \mapsto \alpha^{(t)}$, *and* $\mathbf{z}^{(t)}$ *enters the LSTM.*

$\mathbf{z}^{(t)}$ is a weighted average; uniform weights $1/L$ give the naive model with average pooling.

Soft versus hard attention. Soft attention (above) is differentiable and trained by backpropagation. **Hard attention** samples $s_t \sim \text{Categorical}(\alpha^{(t)})$, $\mathbf{z}^{(t)} = \mathbf{x}_{s_t}$ (one spot per word, vs. soft spreading over bird and water); with no gradient through sampling it needs score-function (REINFORCE) estimators.

2 Spatial and Channel Attention

Weights on the axes of $U \in \mathbb{R}^{H\times W\times C}$:

- **Spatial attention** weights the $H \times W$ locations (Section 1).
- **Channel attention** weights the C channels.

Definition 10.2: squeeze-and-excitation block (SENet)

Given $U = [\mathbf{u}_1, \ldots, \mathbf{u}_C]$ with $\mathbf{u}_c \in \mathbb{R}^{H \times W}$:

$$
\begin{aligned}
s_c &= \frac{1}{HW} \sum_{i=1}^{H} \sum_{j=1}^{W} u_c(i,j) && \text{(squeeze: global average pooling)}, \\
\mathbf{e} &= \sigma\big(W_2 \, \mathrm{ReLU}(W_1 \mathbf{s})\big) && \text{(excitation)}, \\
\tilde{\mathbf{u}}_c &= e_c \, \mathbf{u}_c && \text{(scale)},
\end{aligned}
$$

with $W_1 \in \mathbb{R}^{(C/r) \times C}$ and $W_2 \in \mathbb{R}^{C \times (C/r)}$.

Sigmoid weights $e_c \in (0, 1)$ need not sum to 1. The block adds $2C^2/r$ weights (r e.g. 16), inside residual (SE-ResNet) or mobile (ANTNets) blocks.

3 Inductive Bias

Motivation. Each architecture builds in assumptions; attention has the fewest.

Four families:

- MLP: linear layers and nonlinear functions.
- CNN: convolution, pooling, nonlinearities and an MLP head.
- RNN: MLPs for the recurrent update, plus gating functions.
- Attention and Transformers: dot-product attention plus MLPs.

Data and models can be mixed (1D CNN for sequences, RNN or MLP on pixels).

Definition 10.3: inductive bias

The **inductive bias** of a learner is the set of assumptions it uses to predict outputs for inputs it has not observed.

Mostly **locality** and **invariance**:

- Convolution: 2D grid, translation equivariance (Chapter 7).
- RNN: 1D chain, first-order Markov ($\mathbf{h}_t$ sees the past only via $\mathbf{h}_{t-1}$).

Efficient, but not transferable across modalities; one model for images, point clouds, audio and language (a extbffoundation model, in practice the Transformer) needs few structural assumptions.

4 The General Attention Layer

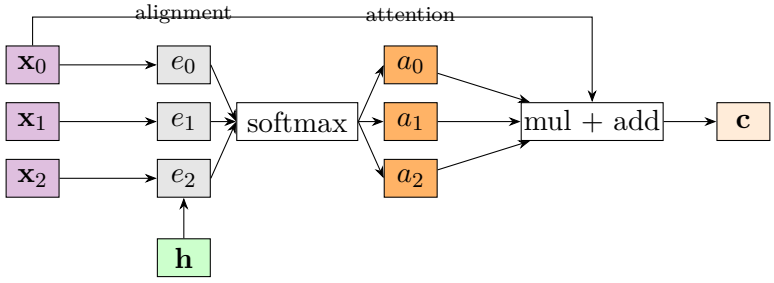
4.1 One query

Definition 10.4: attention with one query

Inputs: vectors $\mathbf{x}_1, \dots, \mathbf{x}_N \in \mathbb{R}^D$ (stacked as $\mathbf{x} \in \mathbb{R}^{N \times D}$) and a query $\mathbf{h} \in \mathbb{R}^D$.

$$\begin{aligned} \text{alignment:} \quad & e_i = f_{\text{att}}(\mathbf{h}, \mathbf{x}_i) = \frac{\mathbf{h} \cdot \mathbf{x}_i}{\sqrt{D}}, \\ \text{attention:} \quad & \mathbf{a} = \text{softmax}(\mathbf{e}), \\ \text{output:} \quad & \mathbf{c} = \sum_{i=1}^{N} a_i \mathbf{x}_i \in \mathbb{R}^D \quad \text{(context vector)}. \end{aligned}$$

An $H \times W \times D$ map gives $N = HW$ vectors. Reordering them leaves the sum $\mathbf{c}$ unchanged (**permutation invariance**).



One query $\mathbf{h}$*: scores, softmax, weighted sum.*

4.2 Why divide by $\sqrt{D}$

For independent entries of $\mathbf{h}, \mathbf{x}$ with mean 0, variance 1, $z = \mathbf{h} \cdot \mathbf{x} = \sum_{m=1}^{D} h_m x_m$ has $\mathbb{E}z = 0$, $\text{Var}\, z = \sum_m \text{Var}(h_m x_m) = D$. Unscaled logits of sd $\sqrt{D}$ make the softmax nearly one-hot, with almost no gradient through the weights; dividing by $\sqrt{D}$ restores unit variance.

4.3 Many queries, keys and values

Definition 10.5: multiple queries

Queries $\mathbf{q}_1, \dots, \mathbf{q}_M \in \mathbb{R}^D$ ($\mathbf{q} \in \mathbb{R}^{M \times D}$), one context vector each:

$$e_{i,j} = \frac{\mathbf{q}_j \cdot \mathbf{x}_i}{\sqrt{D}}, \qquad \mathbf{a}_{\cdot,j} = \text{softmax}(\mathbf{e}_{\cdot,j}), \qquad \mathbf{y}_j = \sum_i a_{i,j} \mathbf{x}_i.$$

$\mathbf{x}$ plays two roles (alignment, output); a separate linear layer for each adds expressiveness and changes dimensions.

Definition 10.6: cross-attention layer

With learnable $W_k \in \mathbb{R}^{D \times D_k}$ and $W_v \in \mathbb{R}^{D \times D_v}$ and queries $\mathbf{q} \in \mathbb{R}^{M \times D_k}$:

$$\mathbf{k} = \mathbf{x} W_k, \quad \mathbf{v} = \mathbf{x} W_v, \quad e_{i,j} = \frac{\mathbf{q}_j \cdot \mathbf{k}_i}{\sqrt{D_k}}, \quad \mathbf{a}_{\cdot,j} = \text{softmax}(\mathbf{e}_{\cdot,j}), \quad \mathbf{y}_j = \sum_i a_{i,j} \mathbf{v}_i \in \mathbb{R}^{D_v}.$$

Definition 10.7: self-attention layer

With $\mathbf{q} = \mathbf{x}W_q$, $W_q \in \mathbb{R}^{D\times D_k}$, the layer maps N inputs to N outputs (**self-attention**); rows as tokens,

$$\mathrm{SA}(\mathbf{x}) = \mathrm{Attn}(\mathbf{x}W_q, \mathbf{x}W_k, \mathbf{x}W_v) = \mathrm{softmax}\Big(\frac{\mathbf{x}W_q(\mathbf{x}W_k)^\top}{\sqrt{D_k}}\Big)\mathbf{x}W_v,$$

softmax by rows: the Transformer's **scaled dot-product attention** $\mathrm{Attn}(Q,K,V) = \mathrm{softmax}(QK^\top/\sqrt{d_k})V$.

$A = \mathrm{softmax}(QK^\top/\sqrt{d_k}) \in \mathbb{R}^{n\times n}$ is row-stochastic, so each output averages all values: a contextual encoding with whole-sequence receptive field (**feature aggregation**).

The four steps. $\mathbf{x} \in \mathbb{R}^{n\times d}$, $W_q, W_k \in \mathbb{R}^{d\times d_k}$, $W_v \in \mathbb{R}^{d\times d_v}$; $Q = \mathbf{x}W_q$ (**query**, "the question"), $K = \mathbf{x}W_k$ (**key**, "the index"), $V = \mathbf{x}W_v$ (**value**, "the information"):

(i) $A = QK^\top \in \mathbb{R}^{n\times n}$, $A_{ij} = \mathbf{q}_i \cdot \mathbf{k}_j$: how much token i uses token j;

(ii) $A' = QK^\top/\sqrt{d_k}$: $\mathrm{Var}\sum_{m=1}^{d_k} q_m k_m = d_k$ as above;

(iii) $\alpha_{ij} = \mathrm{softmax}(A')_{ij}$, row by row;

(iv) $\mathrm{output}_i = \sum_j \alpha_{ij}\mathbf{v}_j$.

Cross versus self. Cross-attention takes queries from elsewhere (a decoder), keys and values from X; self-attention takes all three from X. Captioning (Section 1) is cross-attention (query: sentence state; keys, values: image features). In Definition 10.4 keys and values coincide.

5 Permutation Symmetry

For a permutation matrix P, cross-attention with fixed queries is **permutation invariant** (keys and values reorder together), and self-attention is **permutation equivariant**, $\mathrm{SA}(P\mathbf{x}) = P\ \mathrm{SA}(\mathbf{x})$, since $\mathbf{q}_i \cdot \mathbf{k}_j$ depends only on the two tokens. "Dog bites man" and "man bites dog" agree up to reordering: order must be supplied separately.

6 Positional Encoding

Motivation. Attach a position vector to each token, since self-attention cannot see positions.

$\mathbf{p}_j = \mathrm{pos}(j) \in \mathbb{R}^d$ is concatenated (slides) or added, $\mathbf{x}_j + \mathbf{p}_j$ (Transformer). Options:

1. **Learn a lookup table**: $T \times d$ parameters for $t \in [0, T)$.
2. **Design a fixed function**.

Intuition: in a binary counter each bit is periodic in the position (periods $2, 4, \ldots$).

Definition 10.8: sinusoidal positional encoding

For even d and $k = 0, 1, \ldots, \frac{d}{2} - 1$, let $\omega_k = 1/10000^{2k/d}$ and

$$\mathrm{PE}(t)_{2k} = \sin(\omega_k t), \qquad \mathrm{PE}(t)_{2k+1} = \cos(\omega_k t).$$

Equivalently, $\mathrm{PE}(t) = \big(\sin\omega_0 t, \cos\omega_0 t,\ \ldots,\ \sin\omega_{d/2-1}t, \cos\omega_{d/2-1}t\big)$.

Unlike a table it is defined for every t; frequencies fall geometrically from 1 to about $1/10000$, fast to slow like counter bits.

Example 10.9: $d = 4$ (lecture example)

$\omega_0 = 1$, $\omega_1 = 1/10000^{2/4} = 0.01$: $\mathrm{PE}(t) = (\sin t, \cos t, \sin 0.01t, \cos 0.01t)$ and

$$\mathrm{PE}(0) = (0, 1, 0, 1), \qquad \mathrm{PE}(1) \approx (0.841, 0.540, 0.010, 1.000),$$
$$\mathrm{PE}(2) \approx (0.909, -0.416, 0.020, 1.000).$$

With $\mathbf{z}_j = \mathbf{x}_j + \mathbf{p}_j$ and $\mathbf{x}_{\text{dog}} = (1, 0, 2, 0)$,

$$\mathbf{x}_{\text{dog}} + \mathrm{PE}(0) = (1, 1, 2, 1), \qquad \mathbf{x}_{\text{dog}} + \mathrm{PE}(2) \approx (1.909, -0.416, 2.020, 1.000),$$

so "dog" at positions 0 and 2 differ, and "dog bites man" is no longer a reordering of "man bites dog".

7 Masked Self-Attention

Generation must not see future words. Training on "man plays guitar", the Transformer predicts all next tokens in parallel:

position	input	target
0	`<SOS>`	man
1	man	plays
2	plays	guitar
3	guitar	`<EOS>`

Position 1 could copy its target "plays" from position 2, so query j may attend only to $0, \ldots, j$: future scores are set to $-\infty$ ($e^{-\infty} = 0$).

Definition 10.10: masked (causal) self-attention

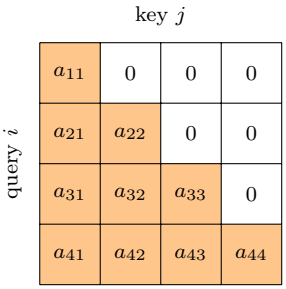
With $\mathbf{q}_i, \mathbf{k}_j, \mathbf{v}_j$ as in Definition 10.7, query i uses the scores

$$e_{i,j} = \begin{cases} \mathbf{q}_i \cdot \mathbf{k}_j / \sqrt{D_k}, & j \le i, \\ -\infty, & j > i, \end{cases} \qquad \mathbf{y}_i = \sum_j a_{i,j} \mathbf{v}_j, \quad \mathbf{a}_{i,\cdot} = \mathrm{softmax}(\mathbf{e}_{i,\cdot}).$$

The slides put the key index first, $e_{i,j} = \mathbf{q}_j \cdot \mathbf{k}_i / \sqrt{D_k}$ ($i \le j$), $-\infty$ ($i > j$):

$$E = \begin{pmatrix} e_{0,0} & e_{0,1} & e_{0,2} \\ -\infty & e_{1,1} & e_{1,2} \\ -\infty & -\infty & e_{2,2} \end{pmatrix}$$

(column j = query j), the transpose of the pattern below. Rows still sum to 1; position i uses only tokens $1, \ldots, i$.



Causal attention: each query sees itself and earlier tokens.

8 Multi-Head Attention

Definition 10.11: multi-head attention

With h heads, each with its own projections $W_q^{(r)}, W_k^{(r)} \in \mathbb{R}^{d\times d_k}$ and $W_v^{(r)} \in \mathbb{R}^{d\times d_v}$,

$$\text{head}_r = \text{Attn}\left(QW_q^{(r)}, KW_k^{(r)}, VW_v^{(r)}\right), \qquad \text{MHA}(Q,K,V) = [\text{head}_1, \ldots, \text{head}_h]\, W_o,$$

with $W_o \in \mathbb{R}^{hd_v\times d}$. Usually $d_k = d_v = d/h$.

Each head (MatMul, Scale, optional Mask, SoftMax, MatMul) can track a different relation (previous word, subject).

9 The Transformer Encoder

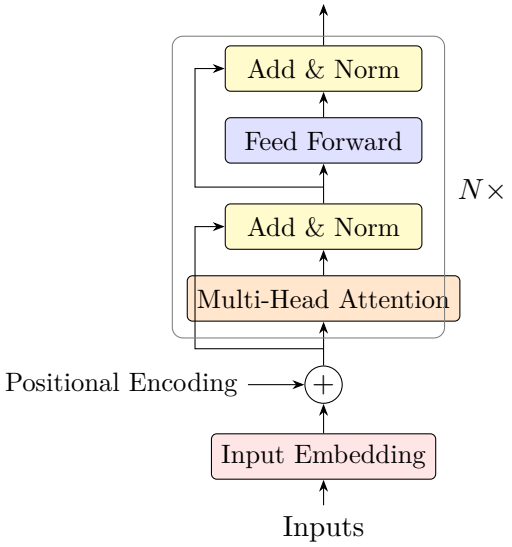
Motivation. Self-attention and position-wise MLPs, each with skip connection and layer norm (Vaswani et al., 2017).

Definition 10.12: Transformer encoder

1. Tokenize and embed: $\mathbf{x} \in \mathbb{R}^{n\times d}$.
2. Add positions: $X_0 = \mathbf{x} + \text{PE}$.
3. Each of N layers:

$$Z = \text{LN}\left(X + \text{MHA}(X,X,X)\right), \qquad X' = \text{LN}\left(Z + \text{FFN}(Z)\right),$$

   $\text{FFN}(\mathbf{z}) = W_2\,\text{ReLU}(W_1\mathbf{z} + \mathbf{b}_1) + \mathbf{b}_2$ per token.
4. Layers are chained.



One encoder layer (boxed), repeated N times; side arrows are skips.

Positions enter only through PE. Cost is dominated by $QK^\top \in \mathbb{R}^{n\times n}$: $O(n^2 d)$ time, $O(n^2)$ memory.

10 Linearization of the Transformer

Motivation. Replace the softmax similarity by a kernel feature map, so attention costs $O(N)$ instead of $O(N^2)$.

Recap. $X = [\mathbf{x}_1, \dots, \mathbf{x}_N]^\top \in \mathbb{R}^{N\times D_x}$.

Step 1. Project: $Q = XW_Q^\top$, $K = XW_K^\top$, $V = XW_V^\top$, with learnable $W_Q, W_K \in \mathbb{R}^{D\times D_x}$, $W_V \in \mathbb{R}^{D_v\times D_x}$; $Q = [\mathbf{q}_1, \dots, \mathbf{q}_N]^\top$, similarly K, V.

Step 2. Output $\hat{V} = [\hat{\mathbf{v}}_1, \dots, \hat{\mathbf{v}}_N]^\top$ with

$$\hat{\mathbf{v}}_i = \sum_{j=1}^{N} \mathrm{softmax}_j\Big(\frac{\mathbf{q}_i^\top \mathbf{k}_j}{\sqrt{D}}\Big)\mathbf{v}_j \iff \hat{V} = \mathrm{softmax}\Big(\frac{QK^\top}{\sqrt{D}}\Big)V = AV.$$

Bottleneck. Storing $A \in \mathbb{R}^{N\times N}$ takes $O(N^2)$ memory and computing AV costs $O(N^2)$ time.

Similarity form. With $s_{ij} = \mathbf{q}_i^\top \mathbf{k}_j/\sqrt{D}$, $\alpha_{ij} = \mathrm{softmax}_j(s_{ij}) = \exp(s_{ij})/\sum_{l=1}^N \exp(s_{il})$, so

$$\hat{\mathbf{v}}_i = \sum_{j=1}^{N} \alpha_{ij}\mathbf{v}_j = \frac{\sum_{j=1}^N \exp(s_{ij})\mathbf{v}_j}{\sum_{l=1}^N \exp(s_{il})} = \frac{\sum_{j=1}^N \mathrm{sim}(\mathbf{q}_i, \mathbf{k}_j)\mathbf{v}_j}{\sum_{j=1}^N \mathrm{sim}(\mathbf{q}_i, \mathbf{k}_j)}, \qquad \mathrm{sim}(\mathbf{q}, \mathbf{k}) := \exp\Big(\frac{\mathbf{q}^\top \mathbf{k}}{\sqrt{D}}\Big).$$

Definition 10.13: linearized (kernel) attention

Replace $\mathrm{sim}(\mathbf{q}_i, \mathbf{k}_j)$ by a kernel that is an inner product of a feature map,

$$k(\mathbf{q}_i, \mathbf{k}_j) = \phi(\mathbf{q}_i)^\top \phi(\mathbf{k}_j), \qquad \phi : \mathbb{R}^D \to \mathbb{R}^r,$$

e.g. $\phi(\mathbf{x}) = \mathrm{ELU}(\mathbf{x}) + 1$ entrywise: $\phi(\mathbf{x})_l = x_l + 1$ $(x_l \geq 0)$, e^{x_l} $(x_l < 0)$. Then

$$\hat{\mathbf{v}}_i^\top \approx \tilde{\mathbf{v}}_i^\top = \frac{\sum_j k(\mathbf{q}_i, \mathbf{k}_j)\mathbf{v}_j^\top}{\sum_j k(\mathbf{q}_i, \mathbf{k}_j)} = \frac{\phi(\mathbf{q}_i)^\top \sum_{j=1}^N \phi(\mathbf{k}_j)\mathbf{v}_j^\top}{\phi(\mathbf{q}_i)^\top \sum_{j=1}^N \phi(\mathbf{k}_j)}.$$

Cost. The sums do not depend on i; set

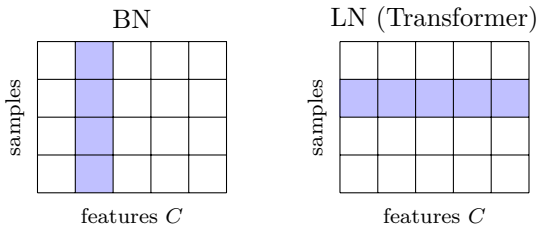
$$S = \sum_{j=1}^{N} \phi(\mathbf{k}_j)\mathbf{v}_j^\top \in \mathbb{R}^{r\times d_v}, \qquad \mathbf{z} = \sum_{j=1}^{N} \phi(\mathbf{k}_j) \in \mathbb{R}^r \quad \Longrightarrow \quad \tilde{\mathbf{v}}_i = \frac{S^\top \phi(\mathbf{q}_i)}{\phi(\mathbf{q}_i)^\top \mathbf{z}}.$$

S costs $O(Nrd_v)$, $\mathbf{z}$ costs $O(Nr)$, and all $S^\top\phi(\mathbf{q}_i)$, $\phi(\mathbf{q}_i)^\top\mathbf{z}$ cost $O(Nrd_v + Nr)$: **linear in** N. In matrix form $A \approx \phi(Q)\phi(K)^\top$ (rows normalized), a product of two thin factors, so $\phi(K)^\top V$ is formed first and the $N \times N$ matrix never appears.

11 Layer Normalization

BN (Chapter 6) uses batch statistics per channel; LN normalizes each sample by itself.

- **BN**: per channel, over batch N and positions H, W.
- **LN in CNNs**: per sample, over C, H, W.
- **LN in Transformers**: per sample and token, over the C features.



Shaded cells share one mean and variance.

Definition 10.14: layer normalization

For $\mathbf{x} \in \mathbb{R}^d$, let $\mu = \frac{1}{d}\sum_k x_k$ and $\sigma^2 = \frac{1}{d}\sum_k (x_k - \mu)^2$. Then

$$\hat{\mathbf{x}} = \frac{\mathbf{x} - \mu\mathbf{1}}{\sqrt{\sigma^2 + \epsilon}}, \qquad \mathrm{LN}(\mathbf{x}) = \boldsymbol{\gamma} \odot \hat{\mathbf{x}} + \boldsymbol{\beta},$$

with learnable $\boldsymbol{\gamma}, \boldsymbol{\beta} \in \mathbb{R}^d$ and a small $\epsilon > 0$.

Example 10.15: BN versus LN (lecture example)

CNN with $B = 2$, $C = 2$, $H = 1$, $W = 2$ ($\epsilon = 0$): channel 1, 2 of $X^{(1)}$ are $(1, 3)$, $(5, 7)$; of $X^{(2)}$, $(9, 11)$, $(13, 15)$.

(i) BN (per channel over the batch): $\mu_1 = \frac{1+3+9+11}{4} = 6$, $\sigma_1^2 = 17$; $\mu_2 = 10$, $\sigma_2^2 = 17$. The first entry 1 of $X^{(1)}$ becomes $(1 - 6)/\sqrt{17}$.

(ii) LN (per image): $X^{(1)}$ has entries $\{1, 3, 5, 7\}$, $\mu^{(1)} = 4$, $(\sigma^{(1)})^2 = 5$; $\mu^{(2)} = 12$, $(\sigma^{(2)})^2 = 5$. The entry 1 becomes $(1 - 4)/\sqrt{5}$.

(iii) LN in a Transformer (per token): token $1 = (1, 5)$ has $\mu = 3$, $\sigma^2 = 4$, so $(1, 5) \mapsto (-1, 1)$; likewise token $2 = (3, 7) \mapsto (-1, 1)$.

12 Why Transformers

Built for translation, the Transformer scales well with data and underlies BERT, GPT-2, Megatron-LM, T5, Turing-NLG (up to 17 billion parameters by early 2020). Its weak inductive bias costs on small data and pays off on large data; the loss keeps falling as the parameter count grows (extbfscaling law). Most large-scale models since (BERT and GPT 2018, GPT-3 2020, ChatGPT 2022, GPT-4 and LLaMA 2023) are Transformers.

13 Vision Transformer

Motivation. An image becomes a sequence of patch tokens, and the Transformer encoder is applied unchanged (Dosovitskiy et al., 2021).

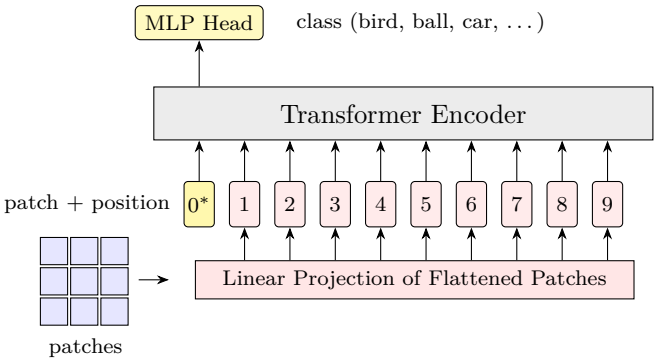
Definition 10.16: Vision Transformer (ViT)

Split an image into N non-overlapping $P \times P$ patches $\mathbf{x}_p^1, \dots, \mathbf{x}_p^N \in \mathbb{R}^{P^2C}$ (tokenization).

1. **Patch embedding**: $\mathbf{x}_p^i E$, $E \in \mathbb{R}^{(P^2C)\times D}$, from the flattened patch (or from ConvNet features: *hybrid*).
2. **Class token**: a learnable $\mathbf{x}_{\text{class}}$ is prepended (the *global token*, as BERT's [class] token).
3. **Position embedding** $E_{pos} \in \mathbb{R}^{(N+1)\times D}$ is added (learnable, or a fixed 2D periodic function).

$$
\begin{aligned}
\mathbf{z}_0 &= [\mathbf{x}_{\text{class}}; \mathbf{x}_p^1 E; \dots; \mathbf{x}_p^N E] + E_{pos}, \\
\mathbf{z}'_\ell &= \mathrm{MSA}\big(\mathrm{LN}(\mathbf{z}_{\ell-1})\big) + \mathbf{z}_{\ell-1}, \qquad \ell = 1, \dots, L, \\
\mathbf{z}_\ell &= \mathrm{MLP}\big(\mathrm{LN}(\mathbf{z}'_\ell)\big) + \mathbf{z}'_\ell, \qquad \ell = 1, \dots, L, \\
\mathbf{y} &= \mathrm{LN}(\mathbf{z}_L^0).
\end{aligned}
$$

LN comes before every block, a residual after every block; the MLP has two layers with GELU. The classifier (MLP head) acts on $\mathbf{y}$, the output of the class token.



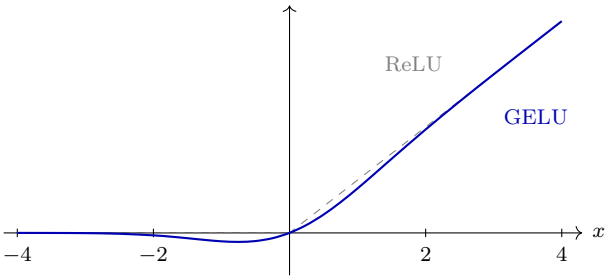
ViT: patch tokens plus the class token 0^*; the head reads only the class token.

13.1 GELU

Definition 10.17: GELU

With Φ the standard normal CDF,

$$\mathrm{GELU}(x) = x\,\Phi(x) \approx 0.5\,x\Big(1 + \tanh\big(\sqrt{2/\pi}\,(x + 0.044715\,x^3)\big)\Big).$$



GELU (solid) is smooth and slightly negative for $x < 0$ (minimum ≈ -0.17 at $x \approx -0.75$); ReLU dashed.

13.2 ViT versus CNN

- **Locality**: CNN features are local; ViT attends globally (non-local).
- **Translation equivariance**: CNN yes; ViT not built in.
- **Fine-tuning at higher resolution**: replace the prediction head and learn it; 2D-interpolate the pre-trained position embeddings.
- **Experiments**: pre-trained on ImageNet, large ViTs are worse than ResNets (BiT); pre-trained on ImageNet-21k or JFT-300M they win, and larger ViTs overtake smaller ones as data grow. In few-shot evaluation ResNets do better on small pre-training sets but plateau sooner.

14 Swin Transformer (Advanced)

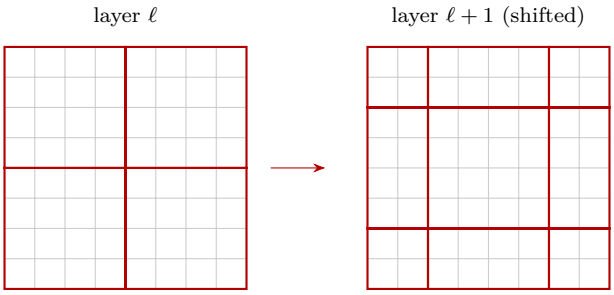
Motivation. Hierarchical feature maps and cost linear in image size, via attention inside local windows (Liu et al., 2021).

For an $h \times w$ map with C channels and $M \times M$ windows,

$$\Omega(\mathrm{MSA}) = 4hwC^2 + 2(hw)^2C, \qquad \Omega(\text{W-MSA}) = 4hwC^2 + 2M^2hwC\ :$$

global attention is quadratic in hw, window attention linear.

- **Patch merging** ($\sim$ pooling): split into 2×2 groups, stack each group depth-wise, combine; the next stage has half the resolution and larger receptive windows (from $4\times$ to $8\times$ to $16\times$ down-sampling).
- **Shifted window**: layer $\ell + 1$ shifts the window grid, so attention crosses the boundaries of the windows of layer ℓ (non-overlapping windows alone never interact).



Self-attention runs inside each red window; shifting the grid links neighbouring windows.

15 Application: Radar-Based Precipitation Nowcasting

- Extreme rainfall is becoming more frequent and intense with climate change (monthly counts of normal, heavy, extreme and very extreme events, 2014 vs 2023).
- Extreme rain is **high-frequency**: small regions with high intensity.
- Model: an encoder–decoder on radar echo sequences ($T \times H \times W$) with 3D (Video) Swin Transformer blocks and patch merging in the encoder, skip connections, cubic dual upsampling in the decoder, a temporal extractor, and the FACL loss.
- Video Swin: attention within windows, shifted windows, and patch merging to larger windows, now over space and time.
- At $t + 10$ min it keeps sharper high-intensity cells than SimVP, Earthformer, AlphaPre and CasCast.